Pontificia Universidad JAVERIANA Colombia

Evaluar aplicaciones con LLMs

Semana 6 · Datasets, evaluadores y LLM-as-judge
Tópicos Avanzados en Analítica II – IA Generativa
Maestría en Analítica para la Inteligencia de Negocios

Sergio Alberto Mora Pardo - H1 2027
GitHub · LinkedIn · Repositorio del curso

Tópicos Avanzados en Analítica II

Estructura del curso

- Fundamentos con LangChain — Semanas 1–3 ✓
- **RAG y evaluación** — Semanas 4–6
- Model Context Protocol — Semana 7
- LangGraph y patrones agénticos — Semanas 8–11
- LLMOps y despliegue — Semana 12

Semana 6 · Evaluar aplicaciones con LLMs

- Del ranking de recuperación a la respuesta completa
- Datasets con respuesta de referencia
- Evaluadores de código
- Un juez con rúbrica (OpenEvals)
- Calibrar el juez contra personas
- Experimentos en LangSmith: calidad, latencia y costo

Las ideas que traemos de la semana 5

Repaso

Antes de avanzar

1. Medimos la recuperación sin modelo: RRF acertó 17 de 19.
2. Una pregunta de seguimiento necesita reformularse antes de recuperar.
3. Cada estrategia cambia el número de llamadas.

¿La nueva versión es mejor?

El problema de la semana

Evidencia, no impresiones

Al cliente de ACME no le llega un puesto en un ranking: le llega **una respuesta**.

Cambiar coseno por RRF, ¿mejora la respuesta, su sustento en fuentes, la latencia y el costo?

Las piezas de una evaluación

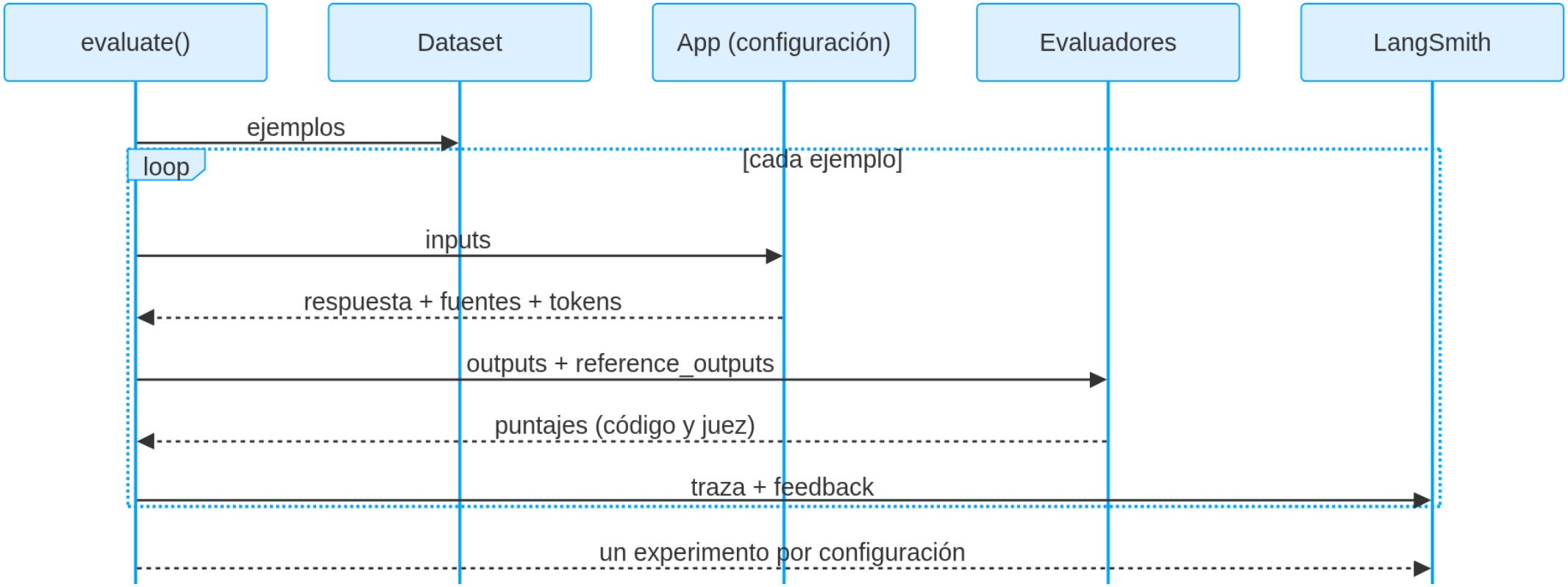
LangSmith y OpenEvals



Un experimento por configuración; la misma entrada y los mismos evaluadores.

Qué hace evaluate()

Diagrama de secuencia



Un experimento. Cada ejemplo pasa por la app y por los evaluadores; LangSmith guarda la traza y los puntajes en un experimento.

Pontificia Universidad JAVERIANA Colombia

Dataset y evaluadores

Qué medimos y cómo

El dataset

datasets/acme_questions.csv → advancedgenai-acme-rag

Campo	Ejemplo
`question`	¿Pueden desembolsar mi crédito en una cuenta de otro banco?
`reference`	No; solo en la cuenta ACME del titular
`source`, `page`	acme_disbursement_policy.pdf, p. 1
Sin respuesta	3 de 22 preguntas: la respuesta correcta es «no está»

Evaluadores de código

L11 · sección 3

Sin modelo, sin costo

Se prueban con salidas falsas antes de usarlos.

```
def retrieval_hit(outputs, reference_outputs):
    if reference_outputs["source"] is None:
        return {"key": "retrieval_hit", "score": None}
    expected = [reference_outputs["source"],
                reference_outputs["page"]]
    return {"key": "retrieval_hit",
            "score": expected in outputs["sources"]}

def citations_ok(outputs, reference_outputs):
    cited = set(re.findall(r"\[(.+\.pdf), p\. (\d+)\]",
                           outputs["answer"]))
    ...  # cada cita debe estar entre lo recuperado
```

Un juez con rúbrica

L11 · sección 4

LLM-as-judge

1. Partimos de `CORRECTNESS_PROMPT` de OpenEvals y lo traducimos con una rúbrica explícita.
2. **Correctness**: misma información que la referencia, sin errores en cifras ni plazos.
3. **Groundedness**: cada afirmación sale de los fragmentos recuperados.
4. Salida estructurada (`score`, `reasoning`); un formato inválido se registra, no rompe la corrida.

Cuatro formas de evaluar

Qué mide cada una

Tipo	Mide bien	Límite
Código	Reglas exactas: página, citas, cifras, formato	No entiende paráfrasis
Juez (LLM-as-judge)	Significado, sustento, completitud	Hay que calibrarlo; cuesta llamadas
Persona	La referencia de lo correcto	Lenta y cara: se usa para calibrar
Comparación por pares	Cuál de dos respuestas es mejor	No da un puntaje absoluto

Calibrar el juez

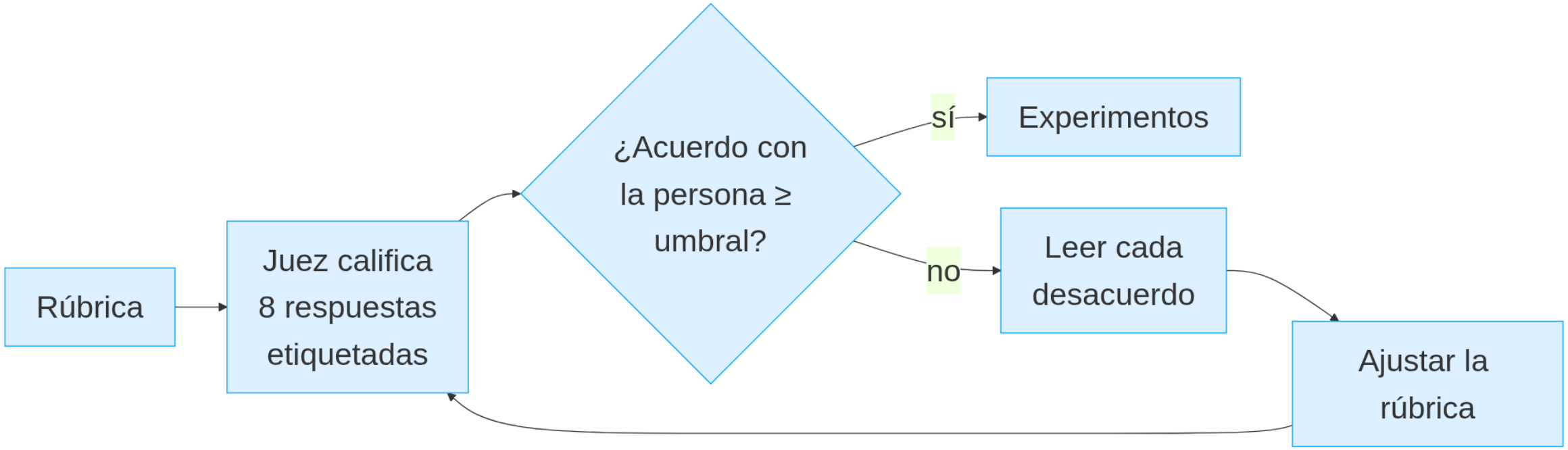
L11 · sección 5

¿Podemos confiar en él?

1. `acme_review_set.csv`: respuestas revisadas por una persona, correctas e incorrectas.
2. El juez califica las mismas respuestas en una traza «L11 · calibración».
3. Medimos el acuerdo juez-persona: en la ejecución guardada fue 100 %.
4. Si el acuerdo es bajo, se corrige la rúbrica antes de evaluar.

El ciclo de calibración

L11 · sección 5



Antes de confiar. Si el acuerdo con la persona no llega al umbral, se corrige la rúbrica y se vuelve a calibrar, antes de los experimentos.

Pontificia Universidad JAVERIANA Colombia

Experimentos

Coseno frente a RRF

Dos experimentos

L11 · sección 6

evaluate

38 llamadas y 11 trazas
con 5 preguntas.

```
from langsmith import evaluate

evaluators = [retrieval_hit, citations_ok,
              correctness, groundedness]

cosine_results = evaluate(
    cosine_app, data=class_examples, evaluators=evaluators,
    experiment_prefix=f"l11-{PARTICIPANT}-coseno",
    metadata={"retriever": "coseno"},
    disable_evaluator_tracing=True,
)
rrf_results = evaluate(rrf_app, ..., experiment_prefix=
    f"l11-{PARTICIPANT}-rrf")
```

Resumen por configuración

L11 · sección 9 · ejecución guardada

Métrica	Coseno	RRF
`retrieval_hit`	0,75	1,00
`citations_ok`	1,00	1,00
`correctness`	0,80	1,00
`groundedness`	1,00	1,00
Latencia media	1,80 s	1,21 s
Costo	US$0,0009	US$0,0007

Leer pocos casos

Cuánto pesa una respuesta

Con 5 preguntas

1. Una sola respuesta mueve el promedio 20 puntos.
2. Reportar siempre el número de preguntas junto al promedio.
3. Mirar las preguntas donde las configuraciones difieren, una por una.
4. Una diferencia es una señal para leer casos; una decisión pide más preguntas.

Qué hay dentro de un experimento

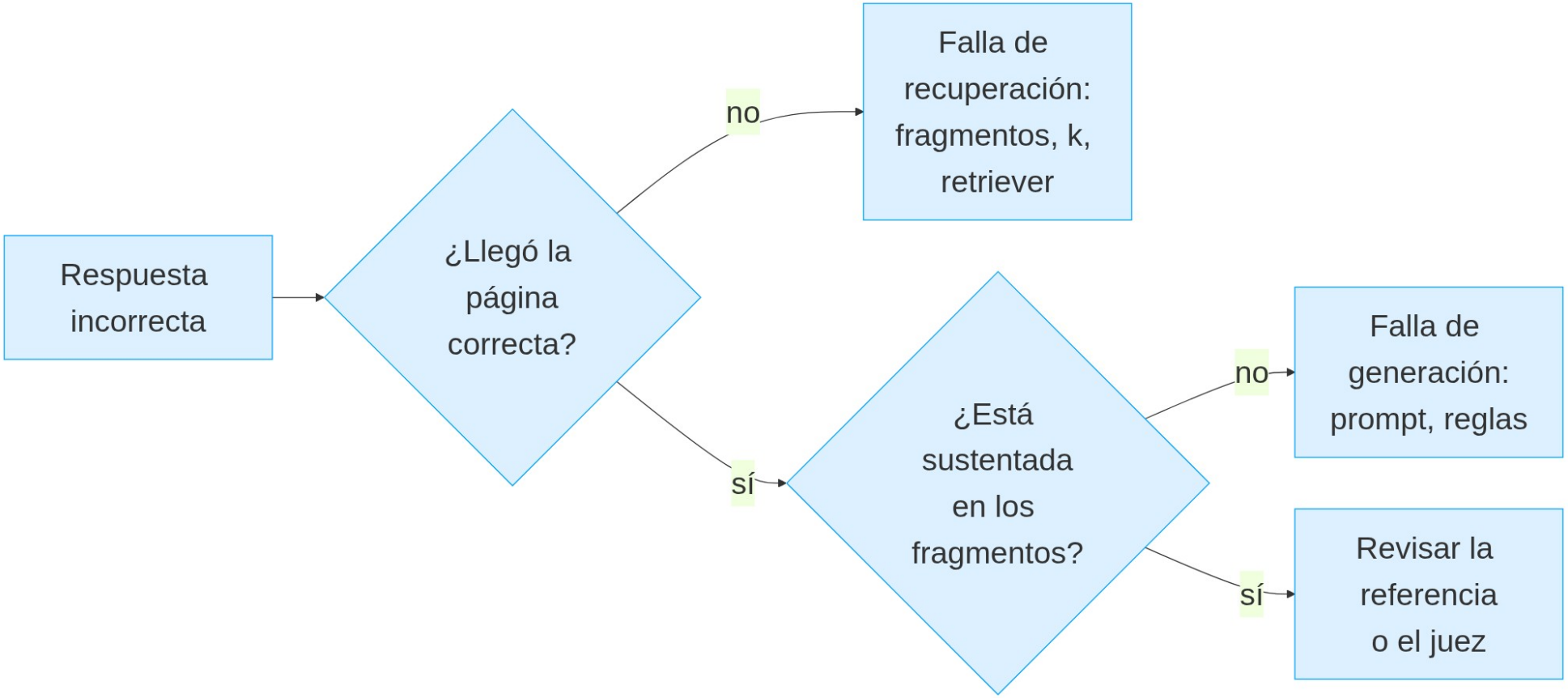
L11 · secciones 7, 8 y 10

Leer la evidencia

1. Cada respuesta es una traza con sus pasos: recuperar y generar.
2. La vista Compare pone las dos configuraciones lado a lado.
3. Revisar por pregunta: dónde gana una y dónde empatan.
4. Separar fallos de formato del juez de las respuestas incorrectas.

¿Recuperación o generación?

Diagnosticar una respuesta incorrecta



Dónde falló. `retrieval_hit` y groundedness separan los dos tipos de falla: cada uno se corrige en un lugar distinto.

Pontificia Universidad JAVERIANA Colombia

Laboratorio

L11-RAGEvaluation

¿Qué haremos?

Laboratorio

Nuestro recorrido

1. Subir el dataset con referencias (una vez por cuenta).
2. Probar los evaluadores de código con salidas falsas.
3. Calibrar el juez contra la revisión humana.
4. Correr los dos experimentos y abrir la vista Compare.
5. Resumir calidad, latencia y costo, y leer las respuestas.

Las ideas que nos llevamos

Cierre

Criterios de construcción

1. Evaluar es comparar configuraciones con la misma entrada.
2. Las comprobaciones de código van primero: son exactas y gratis.
3. Un juez se calibra antes de confiar en él.
4. Calidad, latencia y costo se miran juntos.

Para seguir leyendo

Lecturas de la semana

Fuente	Qué revisar
AI Engineering, cap. 3 y 4	Metodología de evaluación y AI as a judge
Documentación de LangSmith	Datasets, experimentos y Compare
OpenEvals	Rúbricas de correctness y groundedness

Pontificia Universidad JAVERIANA Colombia

Próxima semana

Model Context Protocol: servidores y un agente con sus herramientas.

Notebooks:

`L12-MCPServer.ipynb`

`L13-MCPAgent.ipynb`

Sergio Alberto Mora Pardo - H1 2027

GitHub · LinkedIn · Repositorio del curso